

Cetus CLMM Audit

audited by Asymptotic

September 15, 2025

Summary

Asymptotic conducted a security audit of the Cetus CLMM contract and its MoveSTL and integer-mate dependencies. We identified 8 low-severity issues and made 12 advisories. We identified no medium or higher severity issues. We consider all identified issues to have been adequately addressed.

Audited code

We audited the `cetus_clmm` directory at the initial commit. The final commit reflects the version integrating all fixes:

- initial commit: `https://github.com/CetusProtocol/cetus-for-audit/commit/6b277ca`
- final commit: `https://github.com/CetusProtocol/cetus-contracts/commit/a477627`

Additionally, we audited:

- MoveSTL library at `https://github.com/CetusProtocol/move-stl`
  final commit: `https://github.com/CetusProtocol/move-stl/commit/b1418ed`
- `integer-mate` library at `https://github.com/CetusProtocol/integer-mate`
  final commit `https://github.com/CetusProtocol/integer-mate/commit/09a12d9`

Legend

Issue severity

- **Critical** — Vulnerabilities which allow account takeovers or stealing significant funds, along with being easy to exploit.
- **High** — Vulnerabilities which either can have significant impact but are hard to exploit, or are easy to exploit but have more limited impact.
- **Medium** — Moderate risks with notable but limited impact.
- **Low** — Minor issues with minimal security implications.
- **Advisory** — Informational findings for security/code improvements.

Legal Disclaimer

Terms and Conditions and Liability

This report is subject to the terms and conditions—including liability limitations—established between Asymptotic and the paying entity. By sharing code with Asymptotic, developers acknowledge and agree to these conditions. In the absence of executed agreements, liability is limited to the fees paid for these services.

Scope

This security audit report focuses specifically on reviewing the Move smart contract code. The audit does not analyze or make any claims about other components of the system, including but not limited to:

- Frontend applications and user interfaces
- Backend services and infrastructure
- Off-chain components and integrations
- Deployment procedures and operational security
- Third-party dependencies and external services

Limitations

The findings and recommendations in this report are limited to the Move code implementation and its immediate interactions within the Sui blockchain environment.

L-1: Mixed Version and Emergency Management Creates Control Inconsistencies

Severity: Low

Description

The package version management system serves dual purposes as both a version control mechanism and emergency pause/restore system, creating access control inconsistencies and potential administrative bypasses.

The update_package_version function accepts any u64 value without validation, allowing admins to set emergency versions, downgrade to older versions, or use arbitrary values that could break protocol functionality. It allows AdminCap holders to bypass emergency pause restrictions. While emergency_pause requires ACL_EMERGENCY_PAUSE role, admins can achieve the same effect by setting the version to EMERGENCY_PAUSE_VERSION without proper authorization.

There is no dedicated emergency unpause function. But there is a way to exit emergency pause through update_package_version for AdminCap holders by setting version to any value different from EMERGENCY_PAUSE_VERSION.

Recommendation

Separate emergency controls from version management with dedicated functions and proper access controls:

- Implement specific functions for emergency pause and unpause operations with proper access control
- Enhance the update_package_version function with input validation to prevent setting emergency-specific version numbers through administrative updates. Only allow version increases within reasonable bounds and forbid direct setting of emergency constants.

Remediation

✓ **Remediated**

L-2: Pool Status Management Inconsistency

Severity: Low

Description

The pool has two different pause mechanisms that operate inconsistently, creating potential state synchronization issues. The legacy pause/unpause functions only update the pool.is_pause field, while the newer set_pool_status function updates both pool.is_pause and the granular PoolStatus dynamic field. Legacy unpause can override pool.is_pause regardless what granular settings are.

Additionally, the pause function contains incorrect documentation stating that "unpause are disabled," which is demonstrably false as the unpause function exists and is functional.

Recommendation

Deprecate legacy pause/unpause functions (replace their logic with corresponding abort) and redirect all pause operations through set_pool_status.

In long-term, remove legacy functions and pool.is_pause entirely and establish PoolStatus as the single source of truth for pool operation controls.

Remediation

✓ **Remediated**

L-3: Missing Slippage Protection in remove_liquidity

Severity: Low

Description

The remove_liquidity function lacks slippage protection parameters, making users vulnerable to front-running attacks. Unlike add_liquidity which uses a two-phase design allowing users to reject unfavorable terms, remove_liquidity immediately executes at current pool prices without user control over acceptable token ratios.

Attackers can manipulate pool prices before a user's removal transaction, forcing them to receive unfavorable token ratios, then arbitrage the resulting imbalance for profit.

The most obvious example is with a stablecoin pool (e.g., swapping one stablecoin for another), where a victim attempts to remove liquidity:

- An attacker performs a large swap to artificially shift the stablecoin price away from parity.
- The victim's liquidity removal then executes at the manipulated price, resulting in imbalanced token amounts instead of the expected 1:1 stablecoin ratio.
- The attacker then quickly swaps back to restore price parity, profiting from the arbitrage opportunity created by the victim's unfavorable withdrawal.

Recommendation

Add slippage protection parameters to remove_liquidity:

- min_amount_a: u64
- min_amount_b: u64

Ensure that the calculated amounts are greater than the specified minimum thresholds.

Remediation

✓ **Remediated**

L-4: Missing Package-Level Restriction on Partner Fee Function

Severity: Low

Description

The receive_ref_fee function in the partner module is declared as public, allowing anyone to call this function and add fees to any partner object. While the function itself is financially safe (it only accepts and stores legitimate Balance objects), it creates potential analytics manipulation and operational concerns.

The function is designed for using internally within the protocol by repay_flash_swap_with_partner and repay_flash_loan_with_partner functions in the pool module.

Recommendation

Change the function visibility from public to public(package) to restrict access to only the CLMM package modules.

Remediation

✓ **Remediated**

L-5: Fee Tier Management Lacks Practical Boundaries

Severity: Low

Description

The current tick spacing validation allows values up to TICK_BOUND (443,636), which creates practically unusable pool configurations. High tick spacings result in insufficient price granularity, limiting ticks number across the entire price range.

Also, no maximum limit exists on the number of fee tiers. This enables excessive liquidity fragmentation where the same token pair can have hundreds of separate pools with different tick spacings.

Recommendation

Implement a more restrictive upper bound for tick spacing and fee tier limits based on practical usability requirements.

Remediation

✓ **Remediated** - The addition of new fee tiers configurations are not a permissionless function, which is strictly controlled by admin. This ensures that the number of fee tiers remain manageable and will not lead to uncontrolled fragmentation.

L-6: The get_amount_by_liquidity Lacks Input Validation

Severity: Low

Description

The get_amount_by_liquidity function does not validate that tick_lower ¡ tick_upper, allowing mathematically meaningless calculations. Although validation is implemented within the internal call flow, the function is marked public and remains externally accessible. When called with inverted tick ranges, the function returns seemingly valid amounts without any error indication.

The function uses strict inequality i32::lt instead of inclusive inequality i32::lte for boundary checks. When current_tick == tick_lower, it unnecessarily executes mixed calculations that include get_delta_b, which always equals zero, instead of taking the direct "coin A only" path.

Recommendation

Add assertion ensuring correct ticks order and optimize boundary conditions.

Remediation

✓ **Remediated**

L-7: Rewarder Vault Balance Validation Includes Unclaimed User Rewards

Severity: Low

Description

The update_emission function validates that the vault has sufficient balance for one day of emissions, but incorrectly includes unclaimed user rewards in this calculation. This creates a race condition where emission rate increases can be approved based on rewards that users can immediately claim. The validation logic treats all vault balance as "available for future distribution" when it actually includes earned but unclaimed rewards.

The current system's 24-hour coverage requirement provides minimal buffer for reward claiming scenarios, but this protection becomes meaningless when unclaimed user rewards are included in the balance calculation. Additionally, the First-Come-First-Served distribution model creates inherent unfairness, enabling reward races where early claimers can exhaust the vault, leaving no guarantee that all eligible users will receive their earned rewards.

The position closure mechanism requires all rewards to be claimed before allowing position destruction, creating a critical dependency on vault funding. When reward vaults have insufficient balance, users still can successfully withdraw their entire liquidity and collect all trading fees (no financial loss beyond rewards), but become permanently unable to close their position NFTs. This results in "zombie positions" that remain in user wallets indefinitely.

Recommendation

Implement proper vault balance validation that excludes outstanding reward obligations, or adopt a reserved balance system that tracks committed versus available funds.

Consider extending the minimum collateral requirement beyond one day, or better redesign rewards system to guarantee all users can claim earned rewards.

Remediation

Acknowledged - The contract does not directly record the amount of rewards that have been released but not yet claimed, so it's not possible to perform on-chain validation directly against this value. We address this at the operational level: unclaimed rewards across positions are tracked and calculated off-chain. When updating emission, we're using off-chain data to verify.

L-8: Missing Input Validation for list_p in Skip List Constructor

Severity: Low

Description

The new function in both skip_list and skip_list_u128 modules lacks proper input validation for the list_p parameter. This parameter is critical for the skip list's probabilistic level generation algorithm used in the rand_level function, which performs a modulo operation with list_p.

When list_p = 0, the modulo operation aborts. When list_p = 1, the condition (seed % 1) == 0 is always true, causing every node to be promoted to the maximum level. This breaks the skip list's probabilistic structure and degrades performance from O(log n) to O(n) for search operations.

Recommendation

Add input validation in the new function to ensure list_p ¿ 1.

Remediation

✓ **Commit:** 3372131

A-1: Error Code Collision and Management Issues

Severity: Advisory

Description

The codebase is experiencing issues with error code collisions and unused error codes, which can lead to confusion and maintenance challenges.

Each module independently starts numbering error codes from 0 or 1, leading to numerous instances where error codes overlap (particularly within the range of 1-32). For example, error code 1 is used to represent 9 different errors.

A significant number of error codes are marked as unused, which may be caused by Sui upgradability limitations.

Recommendation

In a long-term, create a centralized error registry/module for managing all error codes and remove unused error codes. This will ensure that each error code is unique and consistently used across the entire codebase, reducing confusion and improving maintainability.

Alternative is implementing module-based pagination by assigning unique error code ranges to each module.

Considering the project is live, well-tested (no active debugging phase), and subject to Sui upgrade limitations, the suitable solution is to define new constants and gradually replace their usage in the code.

However, this approach may only be worth implementing if the team is planning major feature additions that will introduce many new error codes, or in case of a major refactoring. For a stable production system, the current error code collisions represent a code quality issue rather than a functional problem.

Remediation

Acknowledged - Even if we follow the suggestion to centralize all error codes in a single global error module, the unused error codes still cannot be removed due to Sui's upgradability constraints (once an error code is defined and published on-chain, it cannot be deleted in future upgrades). This would likely make the error code system even messier. As there's no security impact from the current situation, so we think it's better not to make changes at this time.

A-2: Misleading API Design in Fetch Functions

Severity: Advisory

Description

The fetch functions across the codebase (fetch_ticks, fetch_positions, fetch_pools) have misleading parameter designs. All three functions accept a vector¡T¿ parameter named start. However, they only check if the vector is empty or use the first element, completely ignoring any additional elements.

Additionally, the functions handle limit = 0 differently, creating unpredictable behavior:

- fetch_pools returns empty vector immediately when limit = 0
- fetch_ticks and fetch_positions loop through entire collections when limit = 0.

Recommendation

Change vector¡T¿ parameters to Option¡T¿ to clearly indicate single starting point support, making the API intention explicit and preventing confusion about multiple elements.

As a short-term solution, include assertions to validate that vectors contain at most one element to keep the current signature and make the code self-explanatory.

Implement consistent zero-limit behavior across all functions.

Remediation

Acknowledged - These are read-only functions that functionally meet our current requirements. They are rarely used in practice, so we do not plan to adjust the parameter design at this stage. Regarding the handling of limit = 0, we will update the implementation to make it consistent across all fetch functions

A-3: Position Display Management Duplication

Severity: Advisory

Description

The position module contains significant code duplication between the init and set_display functions that independently create and manage Display¡Position¿ objects. Both functions duplicate identical hardcoded string literals for display keys and share the same display creation, update, and transfer logic without any code reuse.

Recommendation

Refactor the duplicated code by extracting the hardcoded keys vector as a module-level constant and creating a shared helper function for display creation logic.

Remediation

✓ **Remediated**

A-4: Code Duplication Across Multiple Function Pairs

Severity: Advisory

Description

The codebase contains multiple instances of significant code duplication where function pairs share nearly identical implementations with only minor differences in parameters or access control, creating maintenance overhead and potential for inconsistencies:

- the update_pool_fee and update_flash_loan_fee functions are 100% identical with only a cosmetic difference in parameter naming.
- the collect_protocol_fee and collect_protocol_fee_with_cap functions contain identical core logic for fee extraction, balance splitting, pool state reset, and event emission. The only difference is access control.
- the mint_pool_creation_cap and mint_pool_creation_cap_by_admin also share approximately 90% identical implementation with only minor differences in access control and parameters.

Recommendation

Implement a systematic refactoring approach by creating private helper functions that contain the shared logic:

- update_fee_growth
- collect_protocol_fee_internal
- mint_pool_creation_cap_internal

Conduct a comprehensive codebase review to identify and eliminate additional duplication patterns that may exist.

Remediation

✓ **Remediated**

A-5: Math Functions Duplication

Severity: Advisory

Description

The pool module contains duplicated mathematical functions from the clmm_math module, creating maintenance overhead and inconsistent usage patterns:

- pool::get_amount_by_liquidity function is a 100% identical copy of clmm_math::get_amount_by_liquidit The codebase inconsistently uses both versions.
- pool::get_liquidity_from_amount function is an unused wrapper that simply delegates to clmm_math::get_liquidity_by_amount without adding any functionality.

Recommendation

Remove pool versions of functions entirely.

While Sui upgrade limitations prevent immediate removal, modify pool::get_amount_by_liquidity to simply call clmm_math::get_amount_by_liquidity with the same parameters and use only clmm_math function across codebase.

Remediation

✓ **Remediated**

A-6: Read-Only Functions Uses Mutable Reference

Severity: Advisory

Description

Both get_position_amounts in the pool module and is_allowed_coin in the factory module incorrectly require mutable references despite performing only read operations, creating unnecessary access restrictions and API inconsistency.

Recommendation

Change both function signatures to use immutable references:

- get_position_amounts should accept &Pool¡CoinTypeA, CoinTypeB¿.
- is_allowed_coin should accept &Pools.

This will reflect the read-only nature and improve concurrency.

Remediation

✓ **Remediated** - Regarding the is_allowed_coin function, the &mut Pools parameter is reserved for potential future upgrades, where RegulatedCoin entries may be dynamically added to the denied_list.

A-7: Inconsistent Validation Patterns Allow Wasteful No-Op Transactions

Severity: Advisory

Description

The codebase shows inconsistent patterns for input validation where some functions (e.g. liquidity management or swaps) properly check for zero amounts and edge cases while others allow meaningless no-op operations to proceed.

The collect_protocol_fee, collect_fee and collect_reward functions do not verify if there are actually fees or rewards to collect before proceeding with the operation. In the same way loan functions do not validate loan amount is not zero.

All these functions will emit events and create empty balance objects leading to gas waste and poor user experience.

Recommendation

Add validation in collect_protocol_fee, collect_fee and collect_reward to ensure there are collectible amounts before proceeding.

Add validation in loan functions, that amount is non-zero.

Remediation

✓ **Remediated** - The issue with collect_protocol_fee has been fixed. For collect_fee and collect_reward, we decided not to add zero-amount checks at this stage. In some front-end usage, a single transaction like "Harvest all" may call these functions across multiple positions without pre-validation. To ensure existing workflow to be not interrupted, we will keep the current design for now.

A-8: Inconsistent Configuration Management Creates Deployment Fragility

Severity: Advisory

Description

The codebase uses a mixed approach for configuration defaults where some settings are initialized in contract code while others require external script setup or direct administrative calls, creating deployment order dependencies and architectural inconsistency.

The system initializes protocol fee rates, ACL roles, and rate limits directly in config::init, but leaves fee_tiers as an empty map requiring manual setup via add_fee_tier calls.

This inconsistency is highlighted in init_manager_and_whitelist with hardcoded assumptions about tick spacing 200 existing, creating a brittle deployment sequence.

This mixed approach requires operators to understand and maintain complex deployment sequences where some configuration is automatic (protocol fees, roles) while other critical configuration (fee tiers) must be manually orchestrated in the correct order before certain functions can be called.

Recommendation

First, avoid using hardcoded configuration values like 200 in code by either accepting tick spacing as a function parameter or defining it as a named constant.

For the broader architectural issue, adopt a consistent configuration management approach by moving all essential defaults to single source of truth.

For example, modify the config::init function to create standard fee tiers during deployment, including tick spacing 200 with an appropriate fee rate. This eliminates the deployment order dependency and ensures the required fee tier exists when init_manager_and_whitelist is called. Alternatively, accept tick spacing as a parameter to make the function more flexible.

Remediation

✓ **Remediated**

A-9: Position Snapshot Aggregate Tracking Inconsistency

Severity: Advisory

Description

The position snapshot system used for emergency recovery has an aggregate tracking inconsistency where the add function increments total_value_cut but the remove function fails to decrement it, leading to permanently incorrect aggregate values.

While emergency recovery system is deprecated and unlikely to be used in normal operations (no calls to position_snapshot::remove exist anywhere in the codebase), any future use of the emergency recovery system would result in incorrect aggregate calculations that could affect fund recovery processes or position value assessments.

Recommendation

Fix the aggregate inconsistency or remove/deprecate position_snapshot logic.

Remediation

✓ **Remediated**

A-10: Receipt Structs Inconsistencies

Severity: Advisory

Description

FlashLoanReceipt is the only receipt structure without phantom type parameters, while FlashSwapReceipt and AddLiquidityReceipt both use ¡phantom CoinTypeA, phantom CoinTypeB¿. Although all functions perform additional validation of the pool ID from the receipt, which is sufficient to identify the correct pool, this introduces inconsistency in compile-time type checking and weakens the type safety of FlashLoanReceipt.

Also the AddLiquidityReceipt documentation claims it has fee_amount and ref_fee_amount fields that do not exist in the actual struct.

Recommendation

For type safety consistency, add phantom type parameters to FlashLoanReceipt and update the AddLiquidityReceipt comments.

Remediation

Acknowledged - Partially fixed. Regarding FlashLoanReceipt, although it does not adopt phantom type parameters like other receipts, the current design has no functional, security, or performance risks. Pool validation is already enforced at runtime, and the cost of changing this structure across the system would be significant. For consistency improvements, we may consider unifying receipt designs in a future major upgrade.

A-11: Incomplete Cleanup in Configuration Management

Severity: Advisory

Description

The factory module contains asymmetric cleanup behavior in configuration management functions where addition operations use lazy initialization but removal operations fail to clean up empty containers:

- remove_allowed_pair_config only removes individual tick spacings without checking if the VecSet becomes empty.
- unregister_permission_pair_internal removes individual entries without cleaning up empty inner tables.

Recommendation

Consider implementing proper cleanup logic in both removal functions.

Remediation

✓ **Remediated**

A-12: Redundant Position Info Borrowing

Severity: Advisory

Description

The calculate_and_update_points and calculate_and_update_fee functions follow an inefficient pattern where position::borrow_position_info is called twice - once at the beginning to check liquidity, and again in the else block to retrieve the same information. The initial borrow remains in scope and could be reused, making the second borrow call completely unnecessary.

Both functions store the return values in intermediate variables (points and (fee_a, fee_b)) that serve no purpose other than immediately returning the values.

Recommendation

Optimize both functions by eliminating the redundant position::borrow_position_info calls in the else blocks. Remove the unnecessary intermediate variables by returning directly from the if/else expressions.

Remediation

✓ **Remediated**

About the Auditor

Asymptotic provides a white-glove, formal-verification-based auditing service for Sui smart contracts.

Asymptotic Team

Lead Auditor
Mykhailo Burakhin – Senior Smart Contract Auditor

- Conducted detailed security audits and gas optimizations for complex DeFi applications
- Developed and audited ERC-20, ERC-721, and ERC-1155 token contracts, including staking, vesting, auctions, and launchpads
- Engineered cross-chain bridges using ERC-4337 account abstraction, multi-signature wallets, and upgradeable proxy contracts
- Implemented protections against anti-sniping and front-running, and secure onboarding with MPC and zero-knowledge proofs
- Over 15 years of experience architecting secure, high-performance systems with a strong focus on cryptography, blockchain security, and advanced C++ engineering

Reviewer
Andrei Stefanescu – Chief Scientist

- Led verification of AWS cryptographic algorithms using SAW at Galois
- Created first Ethereum smart contract verification tool
- Three-time International Math Olympiad silver medalist
- PhD in Computer Science from UIUC with David J. Kuck Outstanding Thesis Award
- ACM SIGPLAN Distinguished Paper Award at OOPSLA 2016
- Pioneered K Framework for programming language semantics and verification